

Deliverable #5: Service-Oriented Software Engineering

Part 1: Service Engineering Application

1. Service Candidate Identification

Based on the principles of Service-Oriented Architecture, the following services were identified for the system:

- Item Management Service
 - Classification: Business Service.
 - Justification: This service manages the core entity of the application (lost items). It is highly reusable and independent, as it can be utilized by students searching for lost property, or by security officers managing inventory.
- Claim Management Service
 - Classification: Business Service.
 - Justification: This service handles the lifecycle of item claims. It operates independently to track which student is claiming which item, allowing for clear separation of concerns.
- Claim Resolution Workflow
 - Classification: Coordination Service (Composite).
 - Justification: This service orchestrates the Item and Claim business services to execute a complete real-world process without requiring the client to make multiple manual API calls.

2. Service Requirements Specification

For the Item and Claim Business Services:

- **Functional Requirements:** The system must allow authorized users to Create, Read, Update, and Delete (CRUD) records in the SQL Server database.
- **Non-Functional Requirements:** The API must be stateless, respond within 500ms, and ensure data integrity between parent and child tables.
- **Inputs/Outputs & Constraints:** The services must accept and return data exclusively in application/json format. The system must enforce constraints, such as ensuring an UploadedByUserID is present when creating an item.

3. Service Interface Design

The APIs were designed following RESTful principles :

- **Resources & Endpoints:** /api/items and /api/claims.
- **HTTP Methods:** GET (Retrieve), POST (Create), PUT (Update), and DELETE (Remove).
- **Format:** JSON Request and Response payloads.

Part 2: REST API Implementation & Testing

The following screenshots demonstrate successful Postman tests for the implemented REST APIs.

1. GET APIs (Retrieve Data)

Endpoint: GET /api/items & GET /api/claims

This screenshot shows a Postman workspace with a collection named 'My Collection' containing a request named 'Get data'. The request is a GET method to the endpoint 'http://localhost:8080/lost-and-found-api/api/items'. The 'Headers' tab is active, showing an 'Accept' header with the value 'application/json'. The 'Body' tab is also active, showing a JSON response. The status bar indicates a '200 OK' response with a time of 1.23 s and a size of 1.65 KB. The JSON response is as follows:

```
{
  "itemID": 1,
  "description": "Found near Library entrance",
  "itemtype": {
    "category": "Bag",
    "status": "Pending"
  },
  "datefound": "Feb 26, 2026",
  "title": "Red Backpack",
  "location": "Library Main Entrance",
  "officerID": 0
}
```

On the right side, there is an 'API Testing Assistance' panel with a chat interface for the '#Onboarding-Agent'.

This screenshot shows a Postman workspace with a collection named 'My Collection' containing a request named 'Get data'. The request is a GET method to the endpoint 'http://localhost:8080/lost-and-found-api/api/claims'. The 'Headers' tab is active, showing an 'Accept' header with the value 'application/json'. The 'Body' tab is also active, showing a JSON response. The status bar indicates a '200 OK' response with a time of 427 ms and a size of 460 B. The JSON response is as follows:

```
[
  {
    "claimID": 1,
    "status": "Pending",
    "requestDate": "May 18, 2026, 2:45:52 PM",
    "studentID": 282281,
    "itemID": 10
  },
  {
    "claimID": 2,
    "status": "Approved",
    "requestDate": "May 18, 2026, 2:45:52 PM",
    "studentID": 282285,
    "itemID": 12
  }
]
```

On the right side, there is a 'New Chat' panel with a chat interface for the 'Start using Agent Model'.

2. POST APIs (Create Data)

Endpoint: POST /api/items & POST /api/claims

This screenshot shows a Postman interface for a workspace named "Yassen Hany's Workspace". The left sidebar displays a collection named "My Collection" with two items: "GET Get data" and "Post data". The main panel is set to the "POST" method for the endpoint "http://localhost:8080/lost-and-found-api/api/items". The request body is in "raw" JSON format, containing an object with fields: "title", "description", "category", "location", "status", "officerName", and "officerID". The response status is "201 Created" with a response time of 1.09 s and a size of 407 B. The response body is a JSON object with fields: "itemID", "description", "dateFound", "title", "location", "officerName", and "officerID". A "New Chat" sidebar on the right promotes the "Agent Model".

```
POST http://localhost:8080/lost-and-found-api/api/items

{
  "title": "Black Laptop Charger",
  "description": "Found plugged into the wall in room 382.",
  "category": "Electronics",
  "location": "Room 382",
  "status": "Found",
  "officerName": "Yassin",
  "officerID": 181
}
```

```
{
  "itemID": 0,
  "description": "Found plugged into the wall in room 382.",
  "dateFound": "May 18, 2026, 2:49:44 PM",
  "title": "Black Laptop Charger",
  "location": "Room 382",
  "officerName": "Yassin",
  "officerID": 181
}
```

This screenshot shows a Postman interface for the same workspace. The left sidebar shows the "My Collection" with "GET Get data" and "Post data". The main panel is set to the "POST" method for the endpoint "http://localhost:8080/lost-and-found-api/api/claims". The request body is in "raw" JSON format, containing an object with fields: "studentID", "itemID", and "status". The response status is "201 Created" with a response time of 10 ms and a size of 303 B. The response body is a JSON object with fields: "claimID", "status", "requestDate", "studentID", and "itemID". A "New Chat" sidebar on the right promotes the "Agent Model".

```
POST http://localhost:8080/lost-and-found-api/api/claims

{
  "studentID": 282481,
  "itemID": 15,
  "status": "Pending"
}
```

```
{
  "claimID": 3,
  "status": "Pending",
  "requestDate": "May 18, 2026, 2:51:08 PM",
  "studentID": 282481,
  "itemID": 15
}
```

3. PUT APIs (Update Data)

Endpoint: PUT /api/items/{id} & PUT /api/claims/{id}

The screenshot shows the Postman interface for a PUT request to the endpoint `http://localhost:8080/host-and-found-api/api/items/1`. The request body is a JSON object:

```
{
  "title": "Black Laptop Charger",
  "description": "Found plugged into the wall in room 382.",
  "category": "Electronics",
  "location": "Security Desk",
  "status": "Found",
  "officerName": "Yassin",
  "officerID": 101
}
```

The response is a 200 OK status with a 45 ms response time and 362 B of data. The response body is a JSON object:

```
{
  "itemID": 1,
  "description": "Found plugged into the wall in room 382.",
  "title": "Black Laptop Charger",
  "location": "Security Desk",
  "officerName": "Yassin",
  "officerID": 101
}
```

The screenshot shows the Postman interface for a PUT request to the endpoint `http://localhost:8080/host-and-found-api/api/claims/1`. The request body is a JSON object:

```
{
  "studentID": 202401,
  "itemID": 15,
  "status": "Approved"
}
```

The response is a 200 OK status with a 5 ms response time and 251 B of data. The response body is a JSON object:

```
{
  "claimID": 1,
  "status": "Approved",
  "studentID": 202401,
  "itemID": 15
}
```

4. DELETE APIs (Remove Data)

Endpoint: DELETE /api/items/{id} & DELETE /api/claims/{id}

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8080/lost-and-found-api/api/items/1`. The headers are set to `Accept: application/json` and `Content-Type: application/json`. The response is `204 No Content` with a status of `0/1`. The response body is empty.

Key	Value	Description	Bulk Edit	Presets
✓ Accept	application/json			
✓ Content-Type	application/json			

Body: Cookies Headers 3 Test Results 0/1 204 No Content 81 ms · 112 B · Save Response

Raw Preview Visualize

1

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8080/lost-and-found-api/api/claims/1`. The headers are set to `Accept: application/json` and `Content-Type: application/json`. The response is `204 No Content` with a status of `0/1`. The response body is empty.

Key	Value	Description	Bulk Edit	Presets
✓ Accept	application/json			
✓ Content-Type	application/json			

Body: Cookies Headers 3 Test Results 0/1 204 No Content 3 ms · 112 B · Save Response

Raw Preview Visualize

1

Part 3: Service Composition

Description of Composite Service

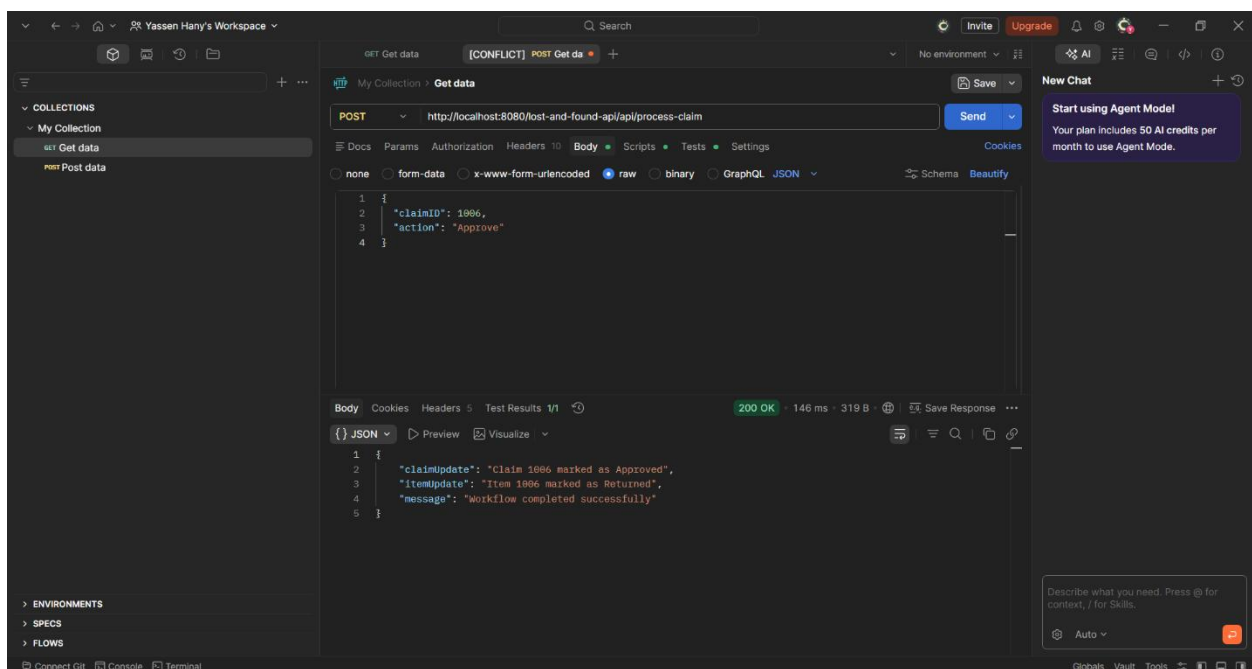
To represent a complete business workflow, a Composite Service was implemented at the endpoint `/api/process-claim`.

Workflow Explanation

When a security officer reviews a claim, updating the claim status alone is insufficient; the system must also secure the physical item. This composite service orchestrates two separate APIs in a single transaction:

1. **API Action 1:** Updates the Claim database record to "Approved" or "Rejected".
2. **API Action 2:** Locates the corresponding Item record and automatically updates its status to "Returned" or "Found".

Composite Service Test



Part 4: API Documentation

The following is the OpenAPI Specification documenting the RESTful services

openapi: 3.0.0

info:

title: Lost and Found API

version: 1.0.0

description: Service-Oriented REST APIs for managing lost items and student claims.

paths:

/api/items:

get:

summary: Retrieve all items

responses:

'200':

description: A list of items

post:

summary: Create a new item

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

title:

type: string

description:

type: string

responses:

'201':

description: Item created successfully

/api/items/{id}:

put:

summary: Update an existing item

parameters:

- in: path

name: id

required: true

schema:

type: integer

requestBody:

required: true

content:

application/json:

schema:

type: object

responses:

'200':

description: Item updated successfully

delete:

summary: Delete an item

parameters:

- in: path

name: id

required: true

schema:

type: integer

responses:

'204':

description: Item deleted successfully

/api/claims:

get:

summary: Retrieve all claims

responses:

'200':

description: A list of claims

post:

summary: Create a new claim

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

studentID:

type: integer

itemID:

type: integer

responses:

'201':

description: Claim created successfully

/api/claims/{id}:

put:

summary: Update an existing claim

parameters:

- in: path

name: id

required: true

schema:

type: integer

requestBody:

required: true

content:

application/json:

schema:

type: object

responses:

'200':

description: Claim updated successfully

delete:

summary: Delete a claim

parameters:

- in: path

name: id

required: true

schema:

type: integer

responses:

'204':

description: Claim deleted successfully

/api/process-claim:

post:

summary: Process a claim (Composite Service)

description: Orchestrates updating a claim status and the corresponding item status.

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

claimID:

type: integer

example: 1006

action:

type: string

example: "Approve"

responses:

'200':

description: Workflow completed successfully

'400':

description: Missing claimID or action

'404':

description: Claim not found